Introduction to Data Structures :→

* These are needed to decide :→
  * (i) How to store data.
  * (ii) How to easily access data. [speed + accuracy]
  * (iii) When to choose which one.

DS — DSA

(Data Structures)

Linear
* Arrays
* Linked Lists
* Stacks
* Queues

(One dimensional or in one line)

Collection (java.util)
{ Choose this when nothing is mentioned }
(Built-in DS in Java)

Non-Linear
* Trees *
* Heaps
* Tries
* Graphs *

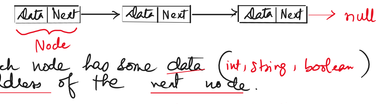
{ Multiple Dimensions }

In C++ it is called STL (Standard Template Library) (#include <bits/stdc++.h>)

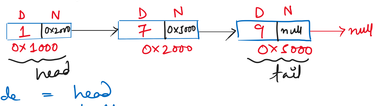
Linked Lists :→ It is a linear data structure containing "nodes". Each node is connected to either a single node or multiple nodes.

Depending on the connection there are three types of linked lists :→

① **Singly Linked Lists :** (Music Playlists / Browsers)



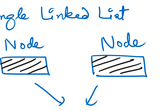
Node :→ Each node has some data (int, string, boolean) & the address of the next node.



Starting Node = head
Ending Node = tail

```
class Node {
(int, char, etc) type-of-data;
→ Node next;
Node (int data) {
    this.data = data;
    this.next = null;
}
}
```

Single Linked List



Same type (Node type)

Initially no connection

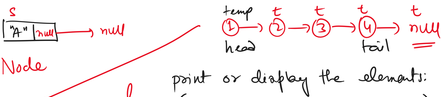
* Add first / head
* Add last / tail
* Remove first / head
* Remove last / tail

it has

* Remove target node.

* Important functions/methods for a Linked List

1. Add first / Add begin / Insert at head
2. Add last / Add end / Insert at Tail
3. Remove first / Remove head / Remove begin
4. Delete last / Delete tail / Delete end
   (Delete / Remove) same
5. Delete / Remove a target value



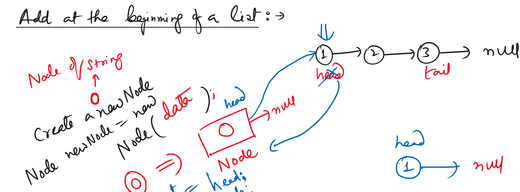
Forward Traversal

Singly Linked List

point or display the elements:
(1 → 2 → 3 → 4 → null)

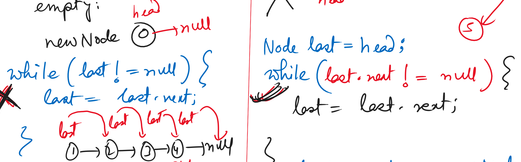
```
Node temp = head;
*while (temp != null) {
    sout (temp.data + " → ");
    temp = temp.next;
}
```

Add at the beginning of a list :→



{Add last / Insert at Tail}

empty:




```
while (last != null) {
    last = last.next;
}
```

```
Node last = head;
while (last.next != null) {
    last = last.next;
}
last.next = newNode;
```